

FINAL COOPERATIVE EDUCATION PRESENTATION

Development of an Enterprise Infrastructure Management System (EIMS)

Ratthabhum Peansukmanee | Student ID: 66011198
Computer Engineering (International Program), Faculty of Engineering
King Mongkut's Institute of Technology Ladkrabang (KMITL)

Host Organization: KANEMATSU ELECTRONICS (THAILAND)
Workplace Supervisor: Mr. Chalernpol Phuangchalam | Academic Supervisor: Asst. Prof. Dr. Phongsak Keeratiwintakorn

Cooperative Education Context & Operational Responsibilities

Host Enterprise Context

- **Context:** Cooperative education placement at KANEMATSU ELECTRONICS (THAILAND)
- **Placement Duration:** Full-time 4th-year Cooperative Education (Academic Year 2026)
- **Workplace Division:** Enterprise Infrastructure Deployment & Systems Support Engineering
- **Supervision:** Under direct technical mentorship of Mr. Chalernpol Phuangchalam

Routine Responsibilities vs. EIMS Capstone

Routine Enterprise Engineering Duties

1. Bare-metal server staging, OS installation, and hardware component testing.
2. Network switch configuration, VLAN tagging, and firewall policy routing.
3. Ad-hoc endpoint troubleshooting and local diagnostic extraction on client servers.

EIMS Capstone Development Scope

1. Engineered central asset records anchored by cryptographic hardware composite fingerprints.
2. Designed an air-gapped, portable USB evidence collection utility for isolated servers.
3. Built an automated ingestion, deduplication, and AI-assisted investigation pipeline.

Problem Statement: Infrastructure Evidence Fragmentation

1. Network Enclave Isolation: High-security production servers reside in air-gapped or segmented enclaves. Continuous streaming daemons cannot cross firewall boundaries, leaving endpoints invisible to central monitoring.

2. Fragile Ephemeral Identity: Dynamic IPs, DHCP leases, and server renames break historical tracking. Infrastructure lacks immutable hardware-anchored identity across server migrations.

3. Labor-Intensive Manual Diagnostics: Engineers must physically access servers and manually execute disparate PowerShell/CMD scripts, producing fragmented, unstructured notes without verification guarantees.

4. High-Volume Telemetry Overload: Windows endpoints generate thousands of routine event logs daily. Triage is slow, prone to oversight, and lacks deterministic prioritization.

5. Absence of Provenance & History: Ad-hoc investigation findings are rarely linked back to specific events, preventing retrospective auditing and historical incident cross-referencing.

Project Objectives (Core Engineering Scope)

OBJ-1: Centralized Asset Registry: Establish central asset records anchored by cryptographic hardware composite fingerprints.

OBJ-2: Portable Disconnected Collection: Implement a portable, read-only collection utility for disconnected Windows hosts requiring no persistent agent installation.

OBJ-3: Ingestion & Normalization Pipeline: Ingest and normalize offline evidence payloads into schema-validated relational records.

OBJ-4: Sequential Deduplication: Enforce application-level sequential SHA-256 deduplication and deterministic severity ranking upon ingestion.

OBJ-5: Assisted AI Analysis: Deploy semantic embeddings and retrieval-augmented generation to provide assistive diagnostic context for prioritized events (bounded at most 10 per import).

OBJ-6: Centralized Investigation Dashboard: Deliver a responsive web interface displaying endpoint profiles, compliance status, event logs, and assisted findings.

OBJ-7: Empirical System Validation: Validate end-to-end data integrity from evidence ingestion to dashboard display across defined evaluation gates.

EIMS System Overview & Technology Stack

Target Endpoints (Windows endpoints; field validation on an authorized Windows Server host): Non-destructive read-only WMI/CIM queries and Windows Event Log `.evtx` channels.

Portable USB Auditor (Embedded CPython 3.14.3 amd64): Bundled runtime executing `Run-EIMS-Audit.bat`; read-only queries with outputs written to configured USB directories.

Core Backend API (FastAPI + Pydantic + Starlette): FastAPI + Pydantic + Starlette backend providing REST APIs and schema validation.

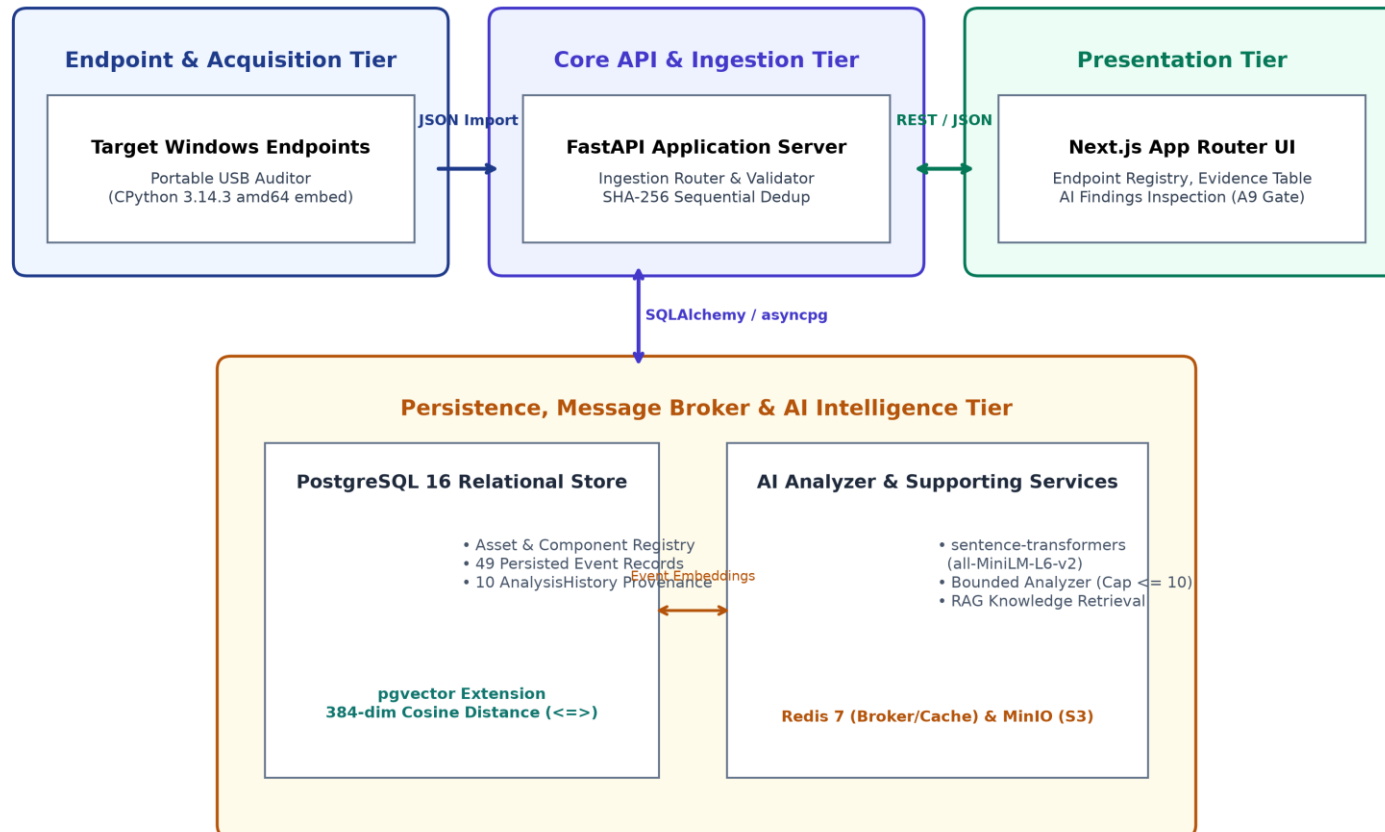
Relational & Vector Store (PostgreSQL 16 + pgvector): ACID transactional relational entity store combined with 384-dimensional vector cosine distance search.

Broker & Caching (Redis 7 & MinIO Object Storage): In-memory caching and message queuing paired with S3-compatible object storage.

AI Analysis Engine (FastEmbed (BAAI/bge-small-en-v1.5) / SentenceTransformer fallback): Bounded assistive analysis generating contextual diagnostic summaries with traceable metadata.

Operator Dashboard (Next.js 14 App Router + Tailwind CSS): Server/Client hybrid rendering dashboard presenting asset details, compliance status, event evidence, and AI findings.

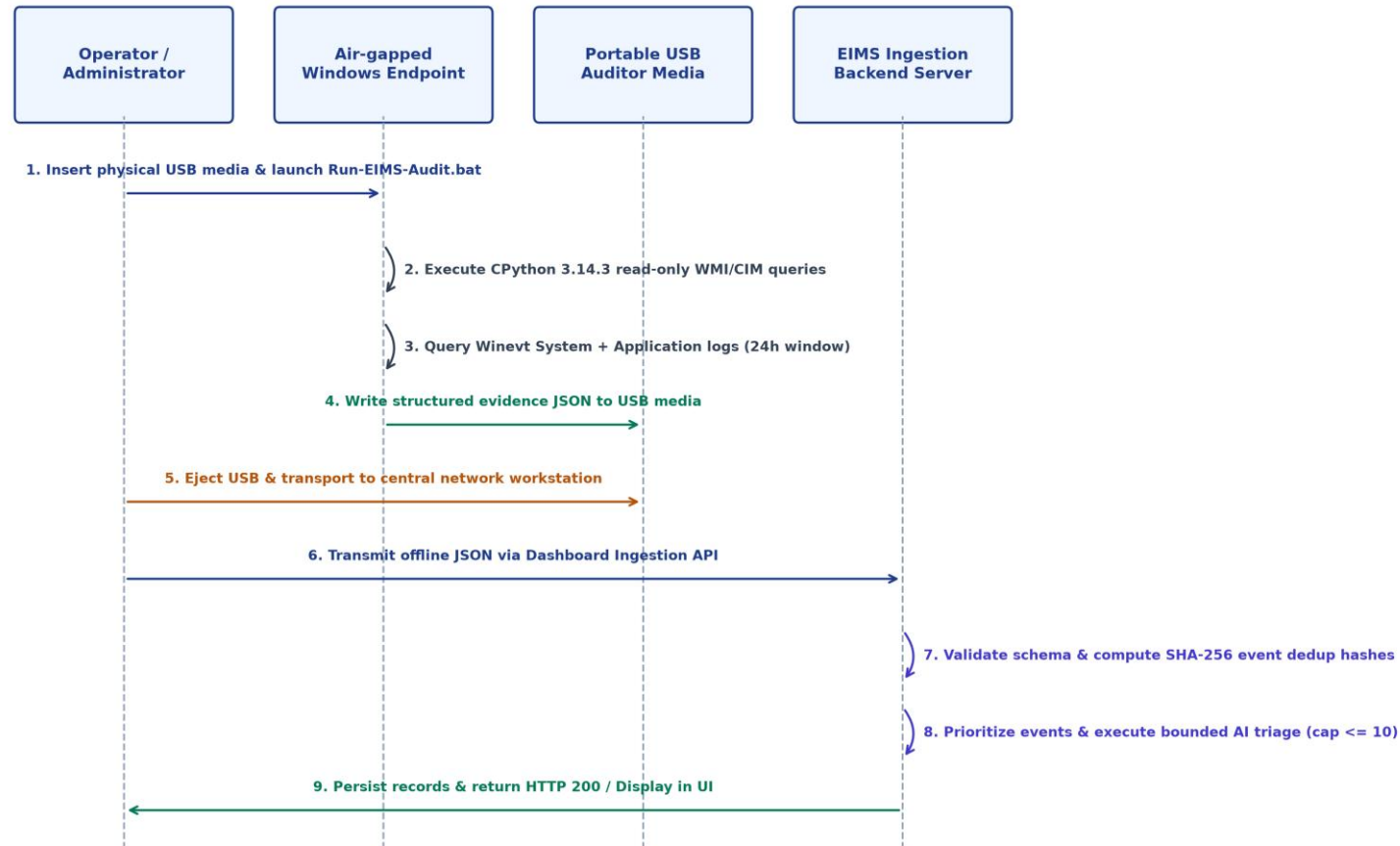
Multi-Tier System Architecture (Hybrid Modular Monolith)



Architectural Highlights

- **Hybrid Modular Monolith**: Consolidates domain services into a cohesive, maintainable codebase without microservice networking overhead.
- **Episodic Disconnected Collection**: USB Auditor bridges network isolation gaps without requiring continuous agent daemons.
- **Bounded Ingestion Pipeline**: Strict schema checks and sequential deduplication help maintain consistent stored data.
- **Persistence & Vector Separation**: Relational tables store entities; pgvector handles mathematical distance operations.

Portable Offline Evidence Collection & Ingestion Sequence



Collection Guarantees

- **Bundled Runtime:**

Runs from USB drive with embedded CPython 3.14.3 runtime; no persistent agent installation required.

- **Read-Only Queries:**

Interrogates WMI/CIM and event logs with non-destructive, read-only queries; writes reports/logs to configured USB directories.

- **Scoped Event Window:**

Captures System and Application logs spanning the preceding 24 hours (default maximum 500 events across combined query).

- **Structured JSON Artifact:**

Generates structured, schema-validated JSON on the USB media prior to ingestion.

Event Ingestion & Application-Level Deduplication

Step 1: Validation

Pydantic payload validation rejecting malformed schemas (HTTP 422).

Step 2: Asset Lookup

Resolve asset by cryptographic hardware composite fingerprint.

Step 3: SHA-256 Dedup

Compute deterministic hash per event and filter duplicates.

Step 4: Persistence

Persist novel event records to PostgreSQL relational table.

Deduplication Hash Formulation

```
DedupKey = SHA-256(asset_id || channel || provider ||  
record_id || occurrence_time)
```

Here, `occurrence\_time` represents the event's recorded timestamp (`TimeCreated`). Because `record\_id` is monotonically assigned per channel by Windows, this hash provides deterministic deduplication across sequential collections of the same event.

Engineering Nuance: Sequential vs. Concurrent

Sequential Application-Level Deduplication:

- The service queries existing keys and filters duplicates in memory prior to database insertion.
- Effectively prevents duplicate records during standard, sequential report imports.
- LIMITATION: Does NOT provide database-level concurrency guarantees. If two identical reports are submitted simultaneously across concurrent threads, race conditions can occur.

Bounded AI-assisted Investigation & Traceable Metadata

Severity Prioritization & Bounded Workload

Deterministic Severity Ranking:

Critical (Level 1) > Error (Level 2) > Warning (Level 3) > Information (Level 4)

Workload Bounding (Evaluated Path ≤ 10):

- AI inference incurs latency and token overhead.
- The evaluated import workflow limits analysis to at most 10 selected new events.
- High-impact failures are triaged first, preventing cognitive overload and pipeline backpressure.

Semantic Embeddings & Conditional Analysis

Semantic Embeddings & Playbook Retrieval:

- Model: FastEmbed `BAAI/bge-small-en-v1.5` primary (384-d dense vectors) with SentenceTransformer `all-MiniLM-L6-v2` fallback.
- Metric: Cosine Distance via pgvector ($\leq$ operator).
- Flow: Checks curated answers first, then retrieval, optional Gemini generation, and fallback.

AnalysisHistory & Assistive Role:

- Persists diagnostic summary, severity score, and recommended action.
- Traceable metadata: Links advice to source event and asset records.
- Assistive Aid: Accelerates human triage; does NOT act as autonomous root-cause authority or auto-remediator.

Empirical Validation Strategy & Multi-Tier Gates

Layer 1: Unit & Ingestion Tests: Sprint 13 test suite (`tests/test\_sprint13\_evidence\_ingestion.py`): 37 reported passing hermetic tests with mocked database and analyzer; separate historical/live PostgreSQL checks.

Layer 2: Frontend Compilation: Strict TypeScript verification (`tsc --noEmit`) and Next.js production build (`next build`) verifying zero type mismatches.

Layer 3: Physical USB Execution: Executed portable package on an authorized Windows Server host verifying read-only WMI/CIM and event log extraction.

Layer 4: Filtered API Verification: Queried backend endpoints (`GET /api/v1/history?asset\_id=...`) verifying HTTP 200 and asset-scoped response payloads.

Layer 5: Browser Runtime (A9 Gate): End-to-end browser inspection validating concurrent rendering of 49 event evidence records and 10 AI findings.

Empirical Results & Reported Field Metrics

37 / 0 / 4

Sprint 13 Test Suite

37 Passed, 0 Failed, 4 Warnings

PASS

Frontend Build

TypeScript & Next.js Production Build

49 Events

Persisted Evidence

34 System + 15 Application (24h)

10 Records

AnalysisHistory

Bounded Workload Path (10 Analyzed)

Field Verification Telemetry Breakdown

- **Evaluated Host Entity:** Anonymized host alias: `ASSET-01` representing enterprise Windows Server environment.
- **Channel Distribution:** 34 System channel events (hardware controller, services, drivers) + 15 Application channel events.
- **Temporal Window:** Preceding 24 hours of operational activity extracted from `%SystemRoot%\System32\Winevt\Logs\`.
- **Filtered API Verification:** `GET /api/v1/history?asset\_id=...` returned HTTP 200 with exactly the 10 asset-linked triage records.
- **Browser Gate A9 Outcome:** PASS — Verified simultaneous visibility of Event Evidence table and AI Findings inspection card. Placeholder 'No AI findings yet' correctly not rendered.

Downstream Live Validation Workflow Trace

ASSET-01

Target Host

49 Events

Persisted in DB

Bounded Analyzer

Cap <= 10

10 Records

AnalysisHistory

HTTP 200

Filtered API

A9 PASS

UI Verification

Critical Distinction: Historical Acquisition vs. Downstream Verification

- **Downstream Verification Scope:**

The latest on-site verification confirmed downstream data integrity: persisted database rows, AnalysisHistory referential linkage, asset-scoped API filtering, and live Next.js UI rendering.

- **Historical Collection Separation:**

Physical USB acquisition, elevated script execution, and raw JSON extraction were proven in earlier field trials and were intentionally NOT re-run during the final downstream gate.

- **Traceability & Auditability:**

By maintaining this clear boundary, the report avoids false claims of full re-execution while demonstrating that persisted evidence correctly propagates to operator interfaces.

Engineering Challenges & Problem-Solving Lessons

Sidebar Hydration Mismatch

Problem: Next.js SSR generated HTML differing from client browser window dimensions, triggering hydration warnings.

Solution: Refactored navigation to two-pass mounting using `useEffect` and CSS container queries.

Lesson: Strict separation of server-rendered layouts from dynamic client-state DOM components.

Filtered History API HTTP 500

Problem: Querying `/api/v1/history?asset\_id=...` failed with HTTP 500 due to a missing `Optional` type import in the route handler.

Solution: Added `from typing import Optional` to the route declaration (commit `acb855f`).

Lesson: Static type annotations and import hygiene must be verified through automated linting and contract checks.

Developer Environment Dataset Variance

Problem: Local developer database was empty and did not contain the field dataset, initially appearing as an application defect.

Solution: Verified environment state and populated local test records to mirror the field dataset structure.

Lesson: Check environment configuration and test data availability before treating empty query results as code bugs.

Benchmark Database Isolation

Problem: Destructive load benchmarking against a shared local development database resulted in accidental test data loss.

Solution: Enforced dedicated test databases and isolated runner environments for benchmarking scripts.

Lesson: Benchmark and evaluation scripts must operate on isolated test instances to prevent accidental data loss.

Scoped Limitations & Future Engineering Roadmap

Acknowledged Scoped Limitations

- Windows-First Focus: Portable collection utility currently targets Windows WMI/winevt subsystems; Linux/macOS collectors deferred.
- System & Application Scope: Scoped strictly to System and Application channels (default max 500 combined events).
- Sequential Dedup Only: Enforces application-level sequential protection; does not provide distributed database concurrency guarantees.
- Bounded Triage Capacity: Evaluated workflow limits analysis to at most 10 prioritized events per batch import.
- Assistive Role: AI suggests diagnoses; does not act as autonomous authority or apply automated patches.
- Academic Prototype: Prototype implementation; not a commercial SIEM or certified forensic suite.

Future Engineering Roadmap (ROADMAP-aligned)

- Cross-Endpoint Correlation: Investigate temporal clustering to detect infrastructure-wide event sequences.
- Database-Level Concurrency: Evaluate database-level constraints for concurrent ingestion safety.
- Infrastructure Scaling: Connection pooling and containerized deployment as operational needs expand.
- Directory Integration: Investigate enterprise single sign-on (SSO) integration with corporate Active Directory.
- Cross-Platform Collection: Study portable collector implementations for Linux systemd and macOS logging.

CONCLUSION & CONTRIBUTIONS

Summary of Project Achievements

1. Centralized Infrastructure Governance: Unified asset tracking with immutable hardware composite fingerprints.
2. Air-Gapped Evidence Acquisition: Demonstrated non-destructive offline collection using embedded CPython 3.14.3 runtimes.
3. Deterministic Deduplication & Prioritization: Implemented sequential SHA-256 event fingerprinting and deterministic triage.
4. Bounded AI Assistance: Delivered RAG-assisted investigation linked to auditable AnalysisHistory records.
5. Validated Downstream Pipeline: Confirmed end-to-end data integrity across 49 field events, HTTP 200 API, and Gate A9.

Thank you. I welcome your questions.